

TDDE01 lab2 report

Zurab Gergaia, Oskar Andersson, Ahmad Soltani

November 2025

Statement of Contribution

- Oskar Andersson did all the work for part 1, Explicit regularization, and answered question 4.1.
- Ahmad Soltani did all the work for part 2, Decision trees and logistic regression for bank marketing, and answered question 4.2.
- Zurab Gergaia did all the work for part 3, Principal components and implicit regularization, and answered question 4.3.

1 Explicit regularization

1.1 Linear regression

Using the 100 channels, this is the resulting linear regression. This uses least squares with no additional penalty.

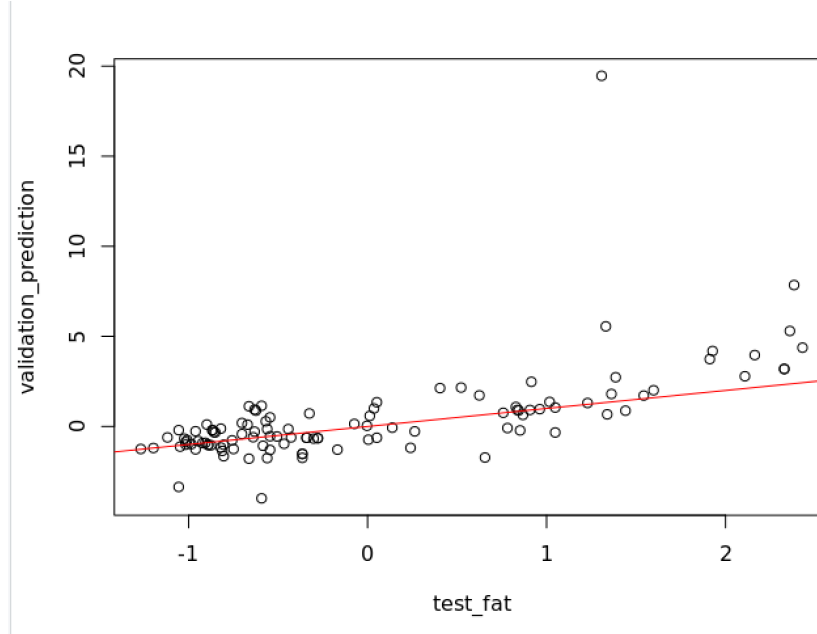


Figure 1: Linear regression

The underlying probabilistic model for a given meat sample is

$$y_i = \beta_0 + \sum_{j=1}^{100} \beta_j x_{ij} + \epsilon_i$$

where β_0 is the intercept, β_j is the coefficient for a specific channel and ϵ is the error.

The training error is 0.005930686 and the validation error is 2.109688 . Since the validation error is much higher than the training error, we can strongly suspect that the model is overfit to the training data. Because of this it is not a particularly reliable model.

1.2 LASSO cost function

The cost function which should be minimized is

$$\frac{1}{n} \sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^{100} \beta_j x_{ij})^2 + \lambda \sum_{j=1}^{100} |\beta_j|$$

We take the mean of the squares of all errors, and then apply an additional penalty which is a constant times the absolute value of the slopes of all channels. This means that the more confident a channel is (i.e the higher β_j) is, the higher the entire cost function becomes. When lambda is high enough, the cost may become so high that the entire channel is ignored. This is how LASSO can feature select - only the features with impact remain.

1.3 Fit LASSO regression to training data

Each line represents a feature (in the case of this example a channel). As lambda decreases along the positive x-axis, the cost function becomes less and less. This lets more channels impact the result, which can visually be seen through more and more features diverging from zero.

If we want to select a model which has only three features, we need to find a place in the plot where only three features are visible. This seems to occur around $\lambda = -\log(2.9) = -0.46$.

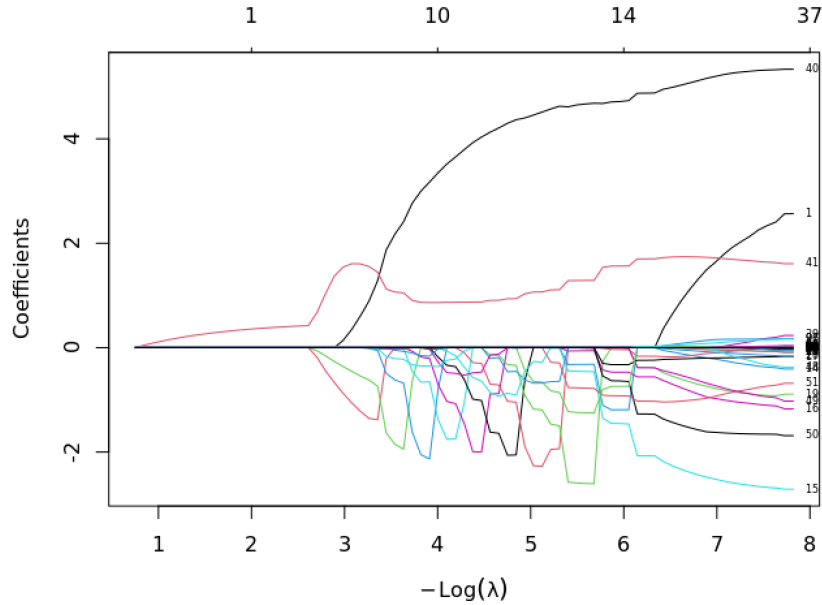


Figure 2: LASSO regression

1.4 Fit Ridge regression to training data

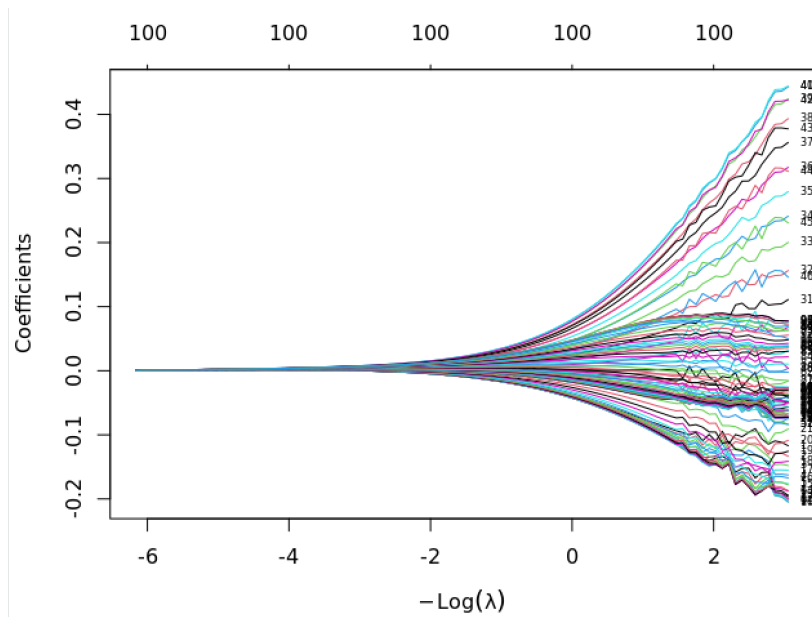


Figure 3: Ridge regression

As ridge regression only allows features to approach zero, not become zero. This means that it does not perform feature selection in the way LASSO does, but it does smoothly lessen the impact of all features. With ridge regression we can mitigate overfitting whilst keeping all features.

1.5 Optimal λ

The optimal value for lambda using 10-fold cross-validation turns out to be $\lambda = 0.004508949$, which visually refers to the right-most dashed line around $x = 5.4$. This is the lowest possible amount of features that will also give the lowest possible mean-squared error.

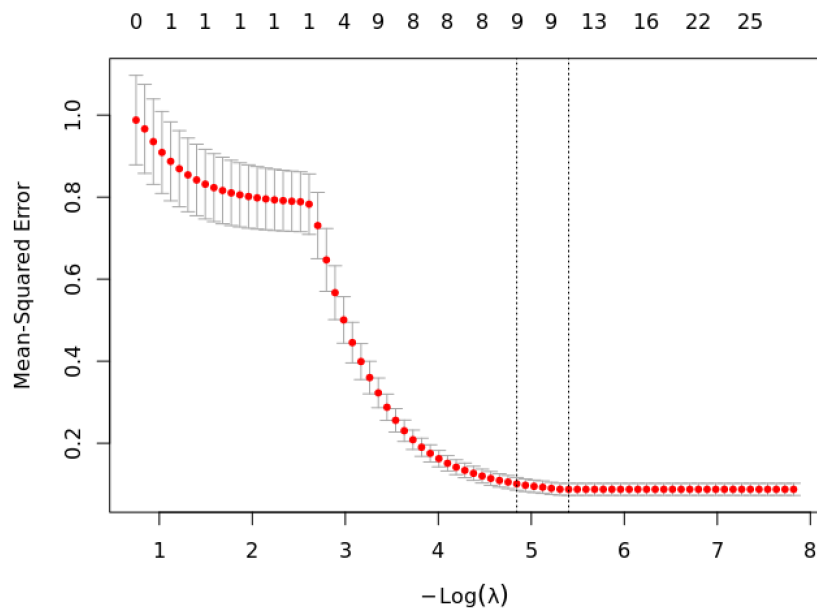


Figure 4: Lasso Model

If we look at 4 on the x-axis, we can see that the corresponding y-value (error) is higher than that of the right-most dashed line. This means that the model will perform worse at that level of lambda.

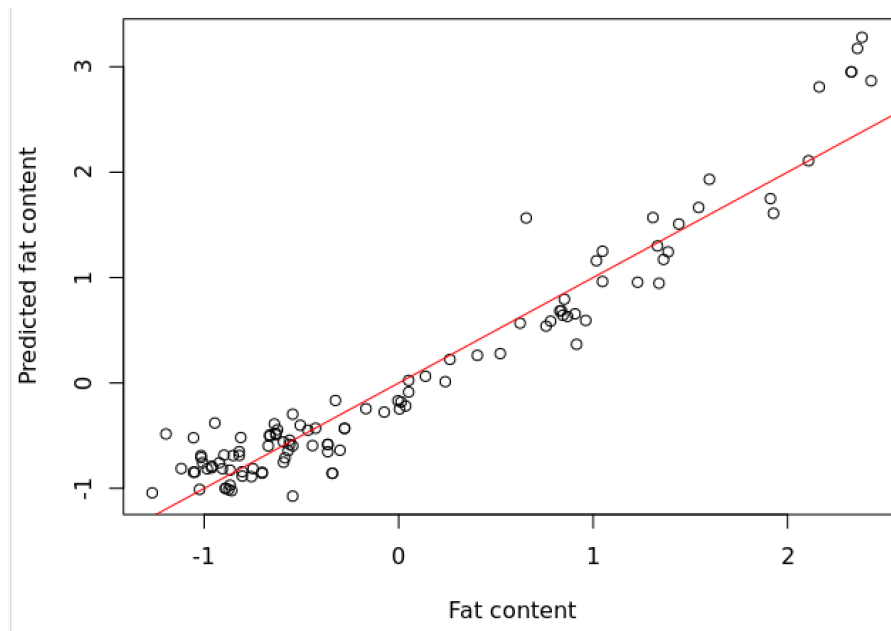


Figure 5: Lasso prediction with optimal lambda vs Actual fat content

As we can see, the actual fat content and the predicted fat content align well to the line $y = x$, which means that the LASSO model works well.

2 Decision trees and logistic regression for bank marketing

2.1 Data Partition

The data is divided into three partitions: 40% for training, 30% for testing, and 30% for validation. The function `sample()` is used to randomly distribute the data samples. Additionally, `set.seed(12345)` ensures reproducibility so that the same random values are generated each time the code runs.

2.2 Fitting the Decision Trees

We fit the trees below using the training dataset and measure the misclassification rates for both the training and validation data.

```
# Misclassification rate for the default tree
mis_train_default <- misclass_rate(tree_default, train)
# The misclassification rate is 0.1048441
```

```
# Misclassification rate for the minsize-controlled tree
```

```

ctrl_minsize <- tree.control(nobs = nrow(train), minsize = 7000)
tree_minsize <- tree(y ~ ., data = train, control = ctrl_minsize)
# The misclassification rate is 0.1048441

# Misclassification rate for the mindev-controlled tree
ctrl_mindev <- tree.control(nobs = nrow(train), mindev = 0.0005)
tree_mindev <- tree(y ~ ., data = train, control = ctrl_mindev)
# The misclassification rate is 0.09400575

```

As shown in the code snippet above, the misclassification rates for the **default tree** and the **minsize-controlled tree** are the same, while the **mindev-controlled tree** has a lower misclassification rate. This occurs because more levels are explored (approximately 122 leaves compared to 5 and 6 leaves). However, a lower misclassification rate does not necessarily indicate the best model, as it may suggest overfitting. To determine which tree performs best, we evaluate the models on the validation dataset:

```

# Misclassification rates on validation data
mis_valid_default <- misclass_rate(tree_default, valid)
# The misclassification rate is 0.1092679

mis_valid_minsize <- misclass_rate(tree_minsize, valid)
# The misclassification rate is 0.1092679

mis_valid_mindev <- misclass_rate(tree_mindev, valid)
# The misclassification rate is 0.1119221

```

After evaluating the models on the validation data, we observe that the misclassification rate for the **mindev-controlled tree** increases significantly from about 9% to around 11%. The misclassification rates for the other two trees remain close to 10%. Based on these results, the best performing trees are the **default tree** and the **minsize-controlled tree** so far.

Changing the value of **minsize** determines whether a tree node is allowed to split. In the case of the **minsize-controlled tree**, each parent node must have at least 7,000 observations before splitting. This constraint makes splitting harder and results in fewer expanded nodes. On the other hand, changing **mindev** sets the minimum decrease in deviance required to accept a split for each node.

For example, changing the model from default (**minsize** = 10) to **minsize** = 7000 results in dropping the variable "housing" and slightly increasing the residual deviance:

- **Default tree:**

Variables used: poutcome, month, contact, housing
 Number of terminal nodes: 6
 Residual mean deviance: 0.6022

- **Minsize-controlled tree:**
 Variables used: `poutcome`, `month`, `contact`
 Number of terminal nodes: 5
 Residual mean deviance: 0.6097

Deviance measures how well the tree’s predicted probabilities agree with the observed yes/no outcomes: it is low when the tree assigns high probability to actual outcomes and high when it is surprised. In our case, the tree with `mindev` = 0.0005 has the lowest training deviance (0.5213 vs. 0.6022 for the default), so it fits the training data best.

2.3 Finding the Optimal Tree

For this part of the assignment, we study the controlled-`mindev` model from the previous assignment with a maximum of 50 leaves. We use the training and validation data to find the optimal depth of the tree. As shown in the figure below, the optimal depth of the tree is 22 leaves, where the tree achieves the lowest validation deviance.

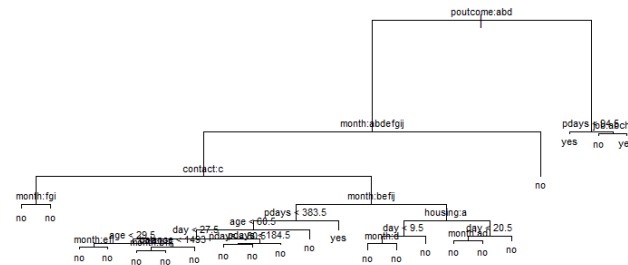


Figure 6: Optimal tree

The tree structure (Figure 6) shows the best tree with a depth of 22 leaves. As shown, the variables `poutcome`, `month`, `pdays`, `contact`, `housing`, `age`, `day`, `balance`, and `job` are the most important variables for decision-making. Some other important information provided by the tree structure includes:

1. `poutcome: abc` → `abc` seems to indicate the options “failure, other, unknown” (go to the left node) and “success” (go to the right node).
2. By `print(tree_best, digits = 3)` we obtain the information `root 18084 12900.0 no (0.8858 0.1142)`, meaning the root node contains all customers in the training data (18,084) and the deviance is 12,900. It also shows that around 88% said “no” and around 11% said “yes” → the prediction is “no”.

3. We can also see that node 2 will be split by the variable `month` and node 3 by the variable `pdays`.

2.4 Confusion Matrix, Accuracy and F1 score

We use a threshold of 0.5, meaning that the tree predicts “yes” if $P(\text{yes} | x) > 0.5$, otherwise it predicts “no”. Considering that the majority of the classes in the test data are “no”, the model predicts “no” most of the time. Thus, the accuracy of the model is high, but the quality of the predictions for the minority class is poor (Figure 7).

	pred	
True	no	yes
no	11872	107
yes	1371	214

Figure 7: Confusion matrix without loss function

To estimate the accuracy and F1 score for the test data using the best tree from the previous assignment, we perform the following:

```
# Accuracy
pred_test <- predict(tree_best, newdata = test, type = "class")
accuracy <- (TP + TN) / (TP + TN + FP + FN)
# The accuracy is 0.8910351

# F1 score
precision <- TP / (TP + FP)
recall <- TP / (TP + FN)
F1 <- 2 * (precision * recall) / (precision + recall)
# The F1 score is 0.224554

# Where:
# TP = ["yes", "yes"]      FN = ["yes", "no"]
# FP = ["no", "yes"]      TN = ["no", "no"]
```

Comment: Accuracy measures the overall proportion of correct classifications, but in this highly imbalanced dataset (89% “no”), accuracy can be high even for models that almost never predict “yes”. On the other hand, the F1 score for the “yes” class focuses on the model’s ability to identify subscribers by combining precision and recall, making it more informative about performance on the minority class.

2.5 Performance with Loss Function

After applying the loss function on the test data, the confusion matrix is as follows:

	pred	
true	no	yes
no	11030	949
yes	771	814

Figure 8: Confusion matrix with loss function

Comment: As shown in the previous part of the assignment (Figure 7), many “yes” cases are not predicted correctly by the model. This is due to the imbalanced dataset and the threshold of 0.5. When the model makes a decision, it predicts many “yes” cases incorrectly as “no”. By adding the cost-sensitive function ($\text{Cost} = 5 \cdot \text{FN} + 1 \cdot \text{FP}$), we change the decision rule so that missing a “yes” prediction becomes more costly for the model. This leads to a lower decision threshold ($\frac{1}{6}$) for predicting “yes”, increasing recall at the expense of more false positives (Figure 8). Compared to the previous results, accuracy slightly decreases from 0.8910351 to 0.8731937, while the F1 score increases from 0.224554 to 0.4862605. The model is now more willing to predict “yes” than before.

2.6 Optimal Tree vs Logistic Regression

The ROC curve plots (Figure 9) the trade-off between True Positive Rate (sensitivity) and False Positive Rate ($1 - \text{sensitivity}$).

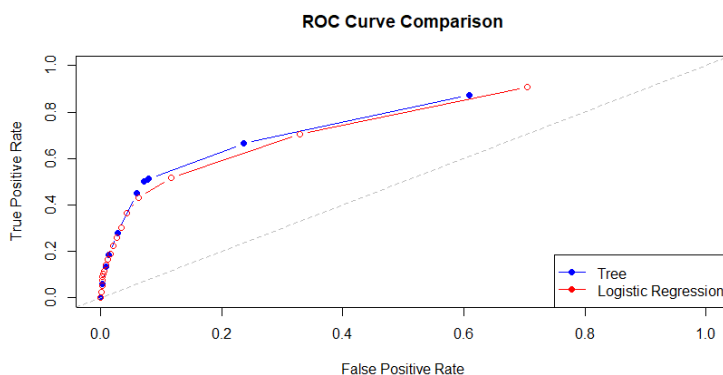


Figure 9: ROC curve

Conclusion: Both models perform much better than the diagonal line (random guessing), indicating that both models have successfully learned predictive patterns from the training data. The optimal tree (blue) seems to rise slightly faster on the left side (Y-axis) than the logistic tree (red). This suggests that the tree might have a slightly higher AUC (Area Under the Curve), or a higher sensitivity to catch positive cases.

Why precision-recall curve could be a better option?: While the ROC curve appears to perform well, it may be too optimistic for imbalanced datasets. Therefore, a precision-recall curve could be more informative about the purity of positive predictions. Precision and recall ignore True Negatives (TN), which prevents the model from looking better than it actually is. Instead, they account for False Positives, meaning that if the model makes false predictions, precision immediately drops.

3 Principal components and implicit regularization

3.1 Implementing PCA

To implement principal component analysis using the *eigen()* function, first we scale all the variables except the target *ViolentCrimesPerPop*. Secondly, we calculate the covariance matrix using the formula $S = \frac{1}{n}X^T X$. Lastly, the covariance matrix is passed to the **eigen** function, which returns eigenvalues and eigenvectors by solving the equation $Su = \lambda u$, where u is the eigenvector and λ is the eigenvalues. The resulting vectors describe the direction of a variance in the "data cloud" and the values describe the magnitude of variance itself. To calculate how many components, we need to obtain at least 95% variance in the data, we calculate the variance ratios of each component using the formula $Ratio(PC_i) = \frac{\lambda_i}{\sum_{j=1}^n \lambda_j}$. Afterwards, we calculate the cumulative sum of ratios and store it. Lastly, we obtain the position where the variance is at least **95%**, which is **35** components in our case. For **PC1** and **PC2** alone, the ratios are **25%** and **16.9%**, respectively.

3.2 PCA analysis using princomp

Figure 10 demonstrates the trace plot for PC1 after it has been analyzed by the *princomp* function. The middle point of the trace on the PC1 axis is 0, which means that the data have been successfully centered and scaled. It resembles a "noise" pattern, which is also a good indication, which tells us that PC1 can be used to describe a variance in the data.

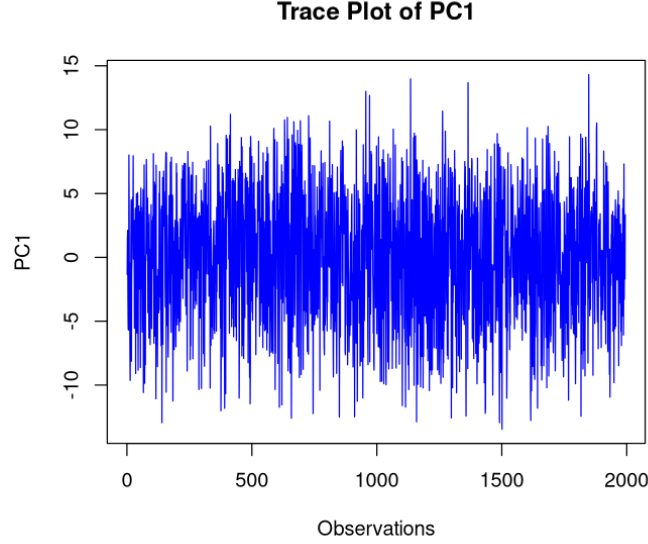


Figure 10: Trace plot of PC1

To obtain notable features of PC1, we can extract the first row of loadings from the pca analysis results and calculate modulo to make the results consistent. Table 1 shows top 5 features. These features are **median family Income**, **median household income**, **percentage of kids in family housing with two parents**, **percentage of households with investment / rent income in 1989** and **percentage of people under the poverty level**. Since all of these features are related to income, we could reason that they are connected and influence each other. For example, if **pctWInvInc** increases, we could expect all other features except **PctPopUnderPov** to increase and **PctPopUnderPov** to decrease instead.

medFamInc	medIncome	PctKids2Par	pctWInvInc	PctPopUnderPov
0.1831478	0.1818171	0.1755406	0.1748076	0.1737183

Table 1: PC1 notable features

Figure 11 is ggplot2 plot, which is colored by **ViolentCrimesPerPop**. Essentially, it demonstrates how PC1 and PC2 relate to the crime level. From the plot we can see, that as PC1 increases on the X axis, the crime level also gradually increases. This tells us that there is a strong correlation between PC1 and the crime level. However, there is no clear color shift along the PC2 axis and hence it captures unrelated variance.

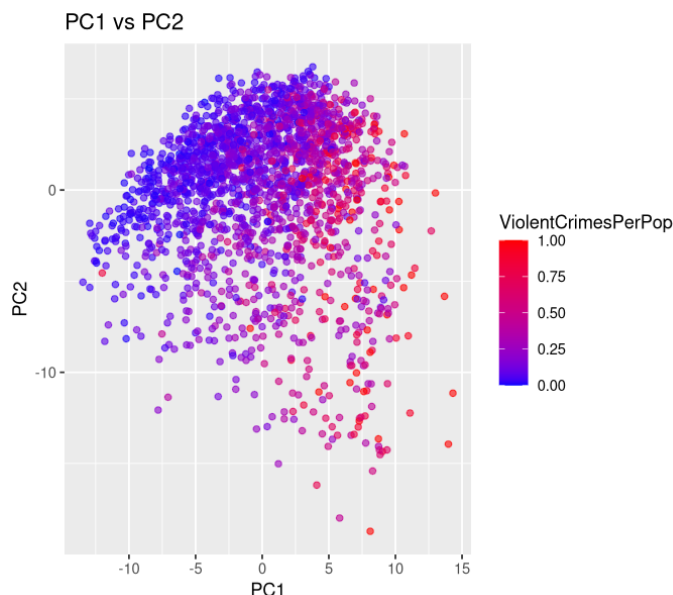


Figure 11: PC1 vs PC2

3.3 Linear regression model

After splitting the data (50/50) and using the *lm* function to estimate a linear model, we get train MSE: **0.275207137480974** and test MSE: **0.424801137490899**. Looking at the model summary, we can see that most of the variables do not contribute to model's accuracy. This is because we include many variables that measure the same thing (e.g **medIncome**, **medFamInc** and **perCapInc** essentially all measuring the income). This causes the model to be quite complex, which leads to overfitting.

3.4 Early stopping criterion

We store **TRAIN_ERRORS** and **TEST_ERRORS** as global variables so that we can monitor their progress while optimizing. Figure 12 is constructed using these data and the associated number of iterations. Approximately by iteration **2000** we see that **test MSE** reaches minimum, but afterwards it keeps increasing until it levels out by the end. The exact iteration when **test MSE** reaches the minimum is **2182**. Table 2 shows a comparison between this model and the model from section **3.3**. As shown, **train MSE** is lower in case of **lm()**, however it is **test MSE** is lower in case of early stopping. This tradeoff is worth it, because at the end of the day, we prefer **test MSE** to be lower.

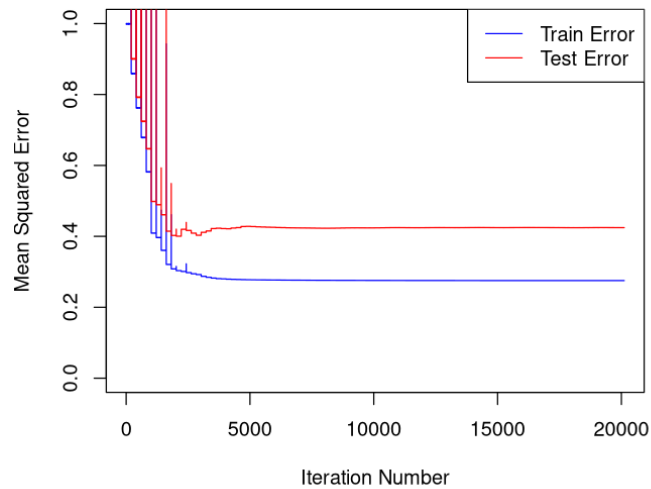


Figure 12: Train and Test Errors throughout training

	Train MSE	Test MSE
Early stopping	0.3032	0.4002
lm()	0.2752	0.4248

Table 2: Early stopping vs lm()

4 Theory

4.1 What are the practical approaches for reducing the expected new data error, according to the book?

Firstly, we need to identify whether the model is suffering from underfitting or overfitting. According to book(pages 76, 293), to reduce overfitting, we can:

- Use a less flexible model, if the model is too complex, it adapts noise in the training data. By choosing a simpler model (e.g fewer parameters, lower-order polynomials).
- Use more regularization(e.g L1 and L2 regularization) to reduce model flexibility
- Utilize an early stopping to stop training before validation errors begin to increase.

- Use bagging, which is a method that reduces the model's variance and hence reduces the generalization gap.
- Collect more training data.

To reduce underfitting, the book suggest(pages 77, 294) following these practical methods:

- Use a more flexible model, which would be able to fit the training data better. This can be achieved by simply tweaking a model's hyperparameter or by replacing an existing model with a more complex one, for example, by a deep neural network.
- Use less regularization to loosen the regularization constraints. this help model to adapt closely to the training data.
- Extend the model's input variables if we suspect that additional input variables carry more information.
- Train the model for a longer period of time to iteratively decrease the training error.

4.2 What important aspect should be considered when selecting minibatches, according to the book?

An important aspect to consider when selecting mini-batches is that they should be balanced and representative of the entire dataset. If the data is sorted by output classes and mini-batches are taken sequentially, some batches might contain only one class, which leads to poor gradient estimates. To avoid this, mini-batches should be formed randomly. A common approach is to shuffle the training data before dividing it into mini-batches, and then reshuffle again after each epoch to maintain randomness and representation.

4.3 Provide an example of modifications in a loss function and in data that can be done to take into account the data imbalance, according to the book

To account for imbalance or asymmetry, you can modify the loss function to assign a higher penalty to specific errors. The book(pages 101-102) provides an example of using **misclassification loss**. If failing to predict $y = 1$ is C times more costly than to misclassify $y = -1$, you can modify the loss function as

$$\text{follows: } L(y, \hat{y}) = \begin{cases} 0 & \text{if } \hat{y} = y \\ 1 & \text{if } \hat{y} \neq y \text{ and } y = -1 \\ C & \text{if } \hat{y} \neq y \text{ and } y = 1 \end{cases}$$

According to the book(page 102) the same effect can be achieved by manipulating the training data itself. You can duplicate all data points belonging to the positive class C times within the training data. By increasing the frequency

of these specific data points, the model will be forced to pay attention to them during training.

A Appendix

A.1 Assignment 1 code

```
library(glmnet)
library(caret)
set.seed(12345)

data <- read.csv("tdde01/2-1/tecator.csv", header= TRUE)
scaler=preProcess(data)
data1=predict(scaler, data)

channels = data1[, 2:101]
fat = data1$Fat
water = data1$Water
protein = data1$Protein

n=dim(channels)[1]
id=sample(1:n, floor(n*0.5))

train_channels=channels[id,]
test_channels=channels[-id,]

train_fat = fat[id]
test_fat = fat[-id]

modeldata = data.frame(fat = train_fat, train_channels)
validationdata = data.frame(fat = test_fat, test_channels)

model <- lm(fat ~ ., data = modeldata)

train_prediction <- predict(model, modeldata)
validation_prediction <- predict(model, validationdata)
plot(validation_prediction)

training_error <- sqrt(mean((train_prediction - train_fat)^2))
validation_error <- sqrt(mean((validation_prediction - test_fat)^2))

training_error
validation_error

#k-fold validation with k = 10 by default
cv_channels = as.matrix(train_channels)
```

```

cv_fat = as.matrix(train_fat)

lasso_model <- cv.glmnet(cv_channels, cv_fat, alpha = 1) # lasso
ridge_model <- cv.glmnet(cv_channels, cv_fat, alpha = 0) # ridge

#LINEAR REGRESSION
plot(test_fat, validation_prediction)
abline(0, 1, col="red")

#RIDGE
model0=glmnet(cv_channels,
              cv_fat, alpha=0,family="gaussian")
plot(model0, xvar="lambda", label=TRUE)

ridge_pred <- predict(ridge_model, as.matrix(test_channels), s = "lambda.min")
plot(test_fat, ridge_pred)
abline(0, 1, col="red")

#LASSO
model1=glmnet(x = cv_channels,
              y = cv_fat, alpha=1,family="gaussian")
plot(model1, xvar="lambda", label=TRUE)

plot(lasso_model)
lasso_model$lambda.min
lasso_model$lambda.1se

coefs <- coef(lasso_model, s = "lambda.min")
sum(coefs != 0) - 1

lasso_pred <- predict(lasso_model, as.matrix(test_channels), s = "lambda.min")
plot(test_fat, lasso_pred, xlab="Fat content", ylab="Predicted fat content")

```

A.2 Assignment 2 code

```

library(readr)
library(tree)

data <- read_delim(
  "data/bank-full.csv",
  delim = ";",
  col_select = -duration,
  show_col_types = FALSE
)

```

```

# Convert ALL character columns to factors in the full data
char_cols <- sapply(data, is.character)
data[char_cols] <- lapply(data[char_cols], factor)

# Split into train / valid / test (40 / 30 / 30)
n=dim(data)[1]
set.seed(12345)
id=sample(1:n, floor(n*0.4))
train=data[id,]
id1=setdiff(1:n, id)
set.seed(12345)
id2=sample(id1, floor(n*0.3))
valid=data[id2,]
id3=setdiff(id1,id2)
test=data[id3,]

#-----Q2-----
train$y <- as.factor(train$y)
valid$y <- as.factor(valid$y)
test$y <- as.factor(test$y)

misclass_rate <- function(model, data) {
  pred_class <- predict(model, newdata = data, type = "class")
  mean(pred_class != data$y)
}

tree_default <- tree(y ~ ., data = train)
mis_train_default <- misclass_rate(tree_default, train)
mis_valid_default <- misclass_rate(tree_default, valid)

ctrl_minsize <- tree.control(nobs = nrow(train), minsize = 7000)
tree_minsize <- tree(y ~ ., data = train, control = ctrl_minsize)
mis_train_minsize <- misclass_rate(tree_minsize, train)
mis_valid_minsize <- misclass_rate(tree_minsize, valid)

ctrl_mindev <- tree.control(nobs = nrow(train), mindev = 0.0005)
tree_mindev <- tree(y ~ ., data = train, control = ctrl_mindev)
mis_train_mindev <- misclass_rate(tree_mindev, train)
mis_valid_mindev <- misclass_rate(tree_mindev, valid)

#-----Q3-----
tree_big <- tree(
  y ~ ., data = train,
  control = tree.control(nrow(train), mindev = 0.0005)
)

```

```

summary(tree_big)
sum(tree_big$frame$var == "<leaf>")

max_leaves <- min(50, sum(tree_big$frame$var == "<leaf>"))
trainScore <- numeric(length(max_leaves))
validScore <- numeric(length(max_leaves))

for (i in 2:max_leaves) {
  pruned_tree <- prune.tree(tree_big, best = i)
  trainScore[i] <- deviance(pruned_tree) / nrow(train)

  pred_tree <- predict(pruned_tree, newdata = valid, type = "tree")
  validScore[i] <- deviance(pred_tree) / nrow(valid)
}

ylim_all <- range(c(trainScore[2:max_leaves], validScore[2:max_leaves]))

plot(2:max_leaves, trainScore[2:max_leaves], type = "b",
     xlab = "Number of leaves", ylab = "Deviance",
     ylim = ylim_all, col = "red")
points(2:max_leaves, validScore[2:max_leaves], type = "b", col = "blue")
legend("topright", legend = c("Training", "Validation"),
      col = c("red", "blue"), pch = 1)

best_i <- which.min(validScore[2:max_leaves]) + 1
print(best_i)
tree_best <- prune.tree(tree_big, best = best_i)
plot(tree_best)
summary(tree_best)
text(tree_best, cex=0.7)
print(tree_best, digits = 3)
#-----Q4-----

pred_test <- predict(tree_best, newdata = test, type = "class")
nrow(test)

cm <- table(True = test$y, pred = pred_test)
cm
TP <- cm["yes", "yes"]
FN <- cm["yes", "no"]
FP <- cm["no", "yes"]
TN <- cm["no", "no"]

accuracy <- (TP + TN) / (TP + TN + FP + FN)
print(accuracy)

```

```

precision <- TP / (TP + FP)
recall <- TP / (TP + FN)
F1 <- 2* (precision * recall) / (precision + recall)
print(F1)

#-----Q5-----
probs_test <- predict(tree_best, newdata = test, type = "vector")
p_yes <- probs_test[, "yes"]

threshold <- 1/6 # L
pred_cost <- ifelse(p_yes > threshold, "yes", "no")

pred_cost <- factor(pred_cost, levels = levels(test$y))
cm_cost <- table(true = test$y, pred = pred_cost)
cm_cost
TP_2 <- cm_cost["yes", "yes"]
FN_2 <- cm_cost["yes", "no"]
FP_2 <- cm_cost["no", "yes"]
TN_2 <- cm_cost["no", "no"]

accuracy_2 <- (TP_2 + TN_2) / (TP_2 + TN_2 + FP_2 + FN_2)
print(accuracy_2)

precision_2 <- TP_2 / (TP_2 + FP_2)
recall_2 <- TP_2 / (TP_2 + FN_2)
F1_2 <- 2* (precision_2 * recall_2) / (precision_2 + recall_2)
print(F1_2)

#-----Q6-----

logistic_model <- glm(y ~ ., data = train, family = "binomial")
probs_glm <- predict(logistic_model, newdata = test, type = "response")
probs_tree <- p_yes
head(probs_glm)

pi_values <- seq(0.05, 0.95, by = 0.05)

tpr_tree <- numeric(length(pi_values))
fpr_tree <- numeric(length(pi_values))
tpr_glm <- numeric(length(pi_values))
fpr_glm <- numeric(length(pi_values))

for (i in 1:length(pi_values)) {
  thresh <- pi_values[i]

```

```

pred_tree <- ifelse(probs_tree > thresh, "yes", "no")
pred_tree <- factor(pred_tree, levels = levels(test$y))
cm_tree <- table(true = test$y, pred = pred_tree)
TP_tree <- cm_tree["yes", "yes"]
FN_tree <- cm_tree["yes", "no"]
FP_tree <- cm_tree["no", "yes"]
TN_tree <- cm_tree["no", "no"]
TPR_tree <- TP_tree / (TP_tree + FN_tree)
FPR_tree <- FP_tree / (TN_tree + FP_tree)

pred_glm <- ifelse(probs_glm > thresh, "yes", "no")
pred_glm <- factor(pred_glm, levels = levels(test$y))
cm_glm <- table(true = test$y, pred = pred_glm)
TP_glm <- cm_glm["yes", "yes"]
FN_glm <- cm_glm["yes", "no"]
FP_glm <- cm_glm["no", "yes"]
TN_glm <- cm_glm["no", "no"]
TPR_glm <- TP_glm / (TP_glm + FN_glm)
FPR_glm <- FP_glm / (TN_glm + FP_glm)

fpr_tree[i] <- FPR_tree
tpr_tree[i] <- TPR_tree
fpr_glm[i] <- FPR_glm
tpr_glm[i] <- TPR_glm
}

plot(fpr_tree, tpr_tree, type = "b", pch = 16, col = "blue",
      xlim = c(0,1), ylim = c(0,1),

      xlab = "False Positive Rate",
      ylab = "True Positive Rate",
      main = "ROC Curve Comparison")
lines(fpr_glm, tpr_glm, type = "b", pch = 1, col = "red")
abline(a=0, b=1, lty=2, col="grey")
legend("bottomright", legend = c("Tree", "Logistic Regression"),
      col = c("blue", "red"), lty=1, pch=16)

```

A.3 Assignment 3 code

```

set.seed(12345)
library(caret)
library(ggplot2)

# ===== Task 1 =====
rawData <- read.csv("communities.csv", header = TRUE)
rawData <- na.omit(rawData)

```

```

dataset <- rawData
dataset[, -ncol(rawData)] <- scale(dataset[, -ncol(rawData)])
dataset <- data.matrix(dataset)

covarianceMatrix <- (t(dataset) %*% dataset) / nrow(rawData)
eigens <- eigen(covarianceMatrix)
zValues <- dataset %*% eigens$vectors[, 1:2, drop = FALSE]

eigenValues <- eigens$values
varianceRatios <- eigenValues / sum(eigenValues)
cumulativeVariance <- cumsum(varianceRatios)
k <- min(which(cumulativeVariance >= 0.95))

print(paste(k, "components needed for 95% accuracy"))
print(paste("Variance of PC1:", eigenValues[1] / sum(eigenValues)))
print(paste("Variance of PC2:", eigenValues[2] / sum(eigenValues)))
# =====

# ===== Task 2 =====
pcaResults <- princomp(dataset)
pc1Scores <- pcaResults$scores[, 1]
pc2Scores <- pcaResults$scores[, 2]

plot(
  pc1Scores,
  type = "l",
  col = "blue",
  main = "Trace Plot of PC1",
  xlab = "Observations",
  ylab = "PC1"
)

pc1Loadings = sort(abs(pcaResults$loadings[, 1]), decreasing = TRUE)
top5Features = head(pc1Loadings, 5)
print("Top 5 features ->")
print(top5Features)

scoresDF <- data.frame(PC1 = pc1Scores,
                       PC2 = pc2Scores,
                       ViolentCrimesPerPop = dataset[, (ncol(dataset))])

ggplot(scoresDF, aes(x = PC1, y = PC2, color = ViolentCrimesPerPop)) +
  geom_point(alpha = 0.6) +
  scale_color_gradient(low = "blue", high = "red") +
  labs(title = "PC1 vs PC2", x = "PC1", y = "PC2")
# =====

```

```

# ===== Task 3 =====
totalRows <- nrow(rawData)
id <- sample(1:totalRows, floor(totalRows * 0.5))
rawTrainData <- rawData[id, ]
rawTestData <- rawData[-id, ]

scaler <- preprocess(rawTrainData)
trainData <- predict(scaler, rawTrainData)
testData <- predict(scaler, rawTestData)
testExpected <- testData[, "ViolentCrimesPerPop"]

model <- lm(ViolentCrimesPerPop ~ ., data = trainData)
summary(model)

trainMSE <- mean(residuals(model)^2)
predictions <- predict(model, newdata = testData)
testMSE <- mean((testExpected - predictions)^2)
print(paste("Train MSE:", trainMSE))
print(paste("Test MSE:", testMSE))
# =====

# ===== Task 4 =====
TRAIN_ERRORS = list()
TEST_ERRORS = list()

OptimCostFn <- function(theta,
                        xTrain,
                        yTrain,
                        xTestValues,
                        yTestValues) {
  trainPredictions <- xTrain %*% theta
  trainError <- mean((yTrain - trainPredictions)^2)

  testPredictions <- xTest %*% theta
  testError <- mean((yTest - testPredictions)^2)

  TRAIN_ERRORS[length(TRAIN_ERRORS) + 1] <- trainError
  TEST_ERRORS[length(TEST_ERRORS) + 1] <- testError
  return(trainError)
}

xTrain = data.matrix(trainData[, -ncol(trainData)])
yTrain = data.matrix(trainData[, ncol(trainData)])
xTest = data.matrix(testData[, -ncol(testData)])
yTest = data.matrix(testData[, ncol(testData)])

```

```

optim(
  matrix(0, ncol = (ncol(xTrain))),
  OptimCostFn,
  xTrain = xTrain,
  yTrain = yTrain,
  xTest = xTest,
  yTest = yTest,
  method = "BFGS"
)

iterations <- 1:length(TRAIN_ERRORS)
plot(
  y = TRAIN_ERRORS,
  x = iterations,
  ylim = c(0, 1.0),
  xlab = "Iteration Number",
  ylab = "Mean Squared Error",
  type = "l",
  col = "blue"
)
lines(y = TEST_ERRORS, x = iterations, col = "red")
legend(
  "topright",
  legend = c("Train Error", "Test Error"),
  col = c("blue", "red"),
  lty = 1
)

optimalIteration <- which.min(TEST_ERRORS)
print(paste("Optimal stopping criterion:", optimalIteration, "iterations"))
print(paste("Train MSE:", TRAIN_ERRORS[optimalIteration]))
print(paste("Test MSE:", TEST_ERRORS[optimalIteration]))
# =====

```